Table 3: **GPT models.** We report the attack success rate according to the GPT-4 judge.

Model	Method	Source	Success rate
GPT-3.5 Turbo	Prompt Automatic Iterative Refinement	Chao et al. (2023)	60%
GPT-3.5 Turbo	Tree of Attacks with Pruning	Zeng et al. (2024)	80%
GPT-3.5 Turbo	Greedy Coordinate Gradient	Zeng et al. (2024)	86%
GPT-3.5 Turbo	Persuasive Adversarial Prompts	Zeng et al. (2024)	94%
GPT-3.5 Turbo	Prompt	Ours	**100%**
GPT-4 Turbo	Prompt Automatic Iterative Refinement	Mazeika et al. (2024)	33%*
GPT-4 Turbo	Tree of Attacks with Pruning	Mazeika et al. (2024)	36%*
GPT-4 Turbo	Tree of Attacks with Pruning (Transfer)	Mazeika et al. (2024)	59%*
GPT-4 Turbo	Prompt	Ours	28%
GPT-4 Turbo	Prompt + Random Search + Self-Transfer	Ours	**96%**
GPT-4o	Prompt	Ours	0%
GPT-4o	Custom Prompt	Ours	72%
GPT-4o	Custom Prompt + Random Search + Self-Transfer	Ours	**100%**

* denotes the numbers from HarmBench (Mazeika et al., 2024) computed on a different set of harmful requests with a judge distilled from GPT-4.

Approach. Similarly to Llama-2-Chat, the standard prompt template, alone or with RS, shows limited effectiveness. However, in contrast to Llama-2-Chat, self-transfer is ineffective here. Motivated by the fact that R2D2 was trained to refuse a specific prompt structure (i.e., request and adversarial suffix), we circumvent safety guardrails by using a different prompt structure. We use an *in-context learning prompt* (see Table 6 in the appendix), which we found the model to be particularly sensitive to. We use random search on top of the in-context prompt to maximize the probability of the initial token "Step" (instead of "Sure") to be consistent with the new prompt template.

Results. As shown in Table 2, using the in-context prompt alone achieves a 90% attack success rate, which random search boosts to 100%. This significantly surpasses the 61% reported by Mazeika et al. (2024) using TAP (Mehrotra et al., 2023). Interestingly, the in-context prompt is less effective on other models like Llama-2-Chat (see Table 22 in the appendix).

4.3 Jailbreaking GPT Models

GPT models remain the most popular LLMs and have non-trivial safety alignment. We consider the following API checkpoints: *gpt-3.5-turbo-1106*, *gpt-4-1106-preview*, and *gpt-4o-2024-05-13*.

Approach. Since December 2023, OpenAI has made the predicted probabilities of their models available via the API, which we leverage for random search. We observed that GPT-3.5 Turbo is extremely brittle to manually designed prompts, with no need for more sophisticated techniques. In contrast, GPT-4 Turbo and GPT-4o demonstrate greater resistance to these adversarial prompt templates. Thus, for these models, we rely on random search and self-transfer to achieve more successful jailbreaks. Additionally, for GPT-4o, we customize the prompt template using the logprobs to guide manual search (i.e., similar to how we developed the main prompt template) and split it into a system and user parts, see Table 7 in Appendix B.2 for details.

Results. Table 3 summarizes our results: with the prompt template alone, we achieve a 100% ASR on GPT-3.5 Turbo, outperforming the baselines. For GPT-4 Turbo, using the prompt alone leads only to a 28% ASR. However, by combining the prompt, random search, and self-transfer, we improve the previous best ASR from 59% to 96%. Our default prompt template is completely ineffective against GPT-4o, resulting in 0% ASR. Moreover, running random search on top of it does not lead to a much higher ASR. However, our custom template (see App. B.2) already leads to 72% ASR, and we get to 100% ASR with random search and self-transfer. This large difference in the ASR again shows the importance of adaptive attacks that are customized for a particular model.

Non-determinism in GPT-3.5/4. The limitation of the API providing only the top-20 log-probabilities is not critical, as it is often straightforward to prompt a desired token, such as "Sure", to appear in the top-20. A more challenging issue is the *non-deterministic* output, since random search does not necessarily have a correct signal to refine the adversarial string. Identical queries can yield varying log-probabilities, as shown in Figure 3, even with a fixed seed and temperature zero in the API. The randomness makes random search less effective, although it still succeeds to a large extent.

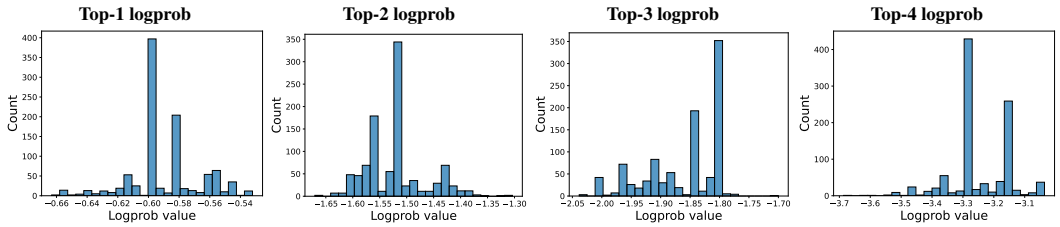

Figure 3: **Non-determinism of GPT models.** The histogram of log-probabilities for the first response token using the same query repeated 1 000 times for GPT-4 Turbo. We use temperature zero and we fix the seed parameter in the API, but the returned log-probs are still non-deterministic.

4.4 Jailbreaking Claude Models

Claude models are known for their high safety levels. In line with this, Anthropic does not provide access to logprobs for these models which prevents direct iterative attack like random search. Thus, we first test a *transfer attack* using an adversarial suffix optimized on GPT-4 with random search. We enhance the attack with multiple random restarts to leverage different generations with temperature one. Then we investigate an attack method that utilizes Anthropic's *prefilling feature* (Anthropic, 2024a), which is not commonly available from other LLM API providers like OpenAI. To improve the attack success rate, we use the prefilling feature together with our prompt (Figure 1) which we split into a system and user part (see Tables 16 and 17 for a detailed ablation).

Transfer attack. As shown in Table 4, the direct transfer attack is especially effective on specific models, such as Claude 3 Haiku, 3 Sonnet, and 3.5 Sonnet, with ASR rates of 98%, 100%, and 96%, respectively. Given the recent release of Claude 3 and Claude 3.5, there are no established baselines for comparison. The attack success rate of the transfer attack improves when the initial segment of the prompt (which corresponds to the set of rules to follow) is provided as the system prompt. In this way, we can achieve 100% ASR on Claude 2.0 and 98% ASR on Claude 3 Haiku. We present an illustrative example of a transfer attack on Claude 3 Sonnet in Figure 5 and show more complete results in the appendix (Table 15). We conclude that while Claude models exhibit increased robustness against static harmful requests, their resistance to adversarial suffixes—challenging to derive without logprobs—is not perfect.

Prefilling attack. The prefilling feature, combined with our prompt template, makes jailbreaking with 100% ASR straightforward on all Claude models—including Claude 3 and 3.5—even without any search, as we show in Table 4. For comparison, the previous best result on Claude 2.0 is 61% (Shah et al., 2023) while we get 100% using only up to 10 random restarts. The Claude 2.1 model appears to be the most robust model in the Claude series and is significantly more robust to both transfer and prefilling attacks. Although we are able to get 100% ASR with 100 restarts, we note that GPT-4, as a semantic judge, sometimes produces false positives, particularly for this model. Complete experimental results, e.g., the number of restarts, are in Tables 16 and 17 in Appendix C.3.

5 Adaptive Attacks for Trojan Detection

Finding universal trojan strings in poisoned models—short suffixes that can be added to any input—is nearly identical to the standard jailbreaking setting. Here we describe our winning solution for the trojan detection competition hosted at SatML 2024. For more details about other solutions and the competition setup, we refer to Appendix B.3 and to the competition report (Rando et al., 2024).

Setup. Rando & Tramèr (2023) showed the possibility of implanting backdoor attacks during the RLHF training of LLMs (Ouyang et al., 2022) by poisoning a small percentage of the preference data with a universal suffix. Then a model that typically refuses to answer harmful queries can then be jailbroken by appending the suffix to any request. Rando & Tramèr (2024) recently launched a competition to retrieve backdoor attacks in five Llama-2-7B models, each poisoned with a different trojan. A reward model was also provided to evaluate the safety of prompt-response pairs (higher scores to safer responses), alongside a dataset of harmful requests. The goal is to discover triggers (5 to 15 tokens long) acting as universal jailbreaks for each model.

Approach. Random search could be directly applied to optimize the score provided by the reward model on some training examples. However, despite the triggers being relatively short, the search

Table 4: **Claude models.** We report the attack success rate according to the GPT-4 judge.

Model	Method	Source	Success rate
Claude 2.0	Persuasive Adversarial Prompts	Zeng et al. (2024)	0%
Claude 2.0	Greedy Coordinate Gradient	Chao et al. (2023)	4%
Claude 2.0	Prompt Automatic Iterative Refinement	Chao et al. (2023)	4%
Claude 2.0	Persona Modulation	Shah et al. (2023)	61%$^{\alpha}$
Claude 2.0	Prompt + Transfer from GPT-4	Ours	**100%**
Claude 2.0	Prompt + Prefilling Attack	Ours	**100%**
Claude 2.1	Foot-in-the-Door Attack	Wang et al. (2024)	68%$^{\beta}$
Claude 2.1	Prompt + Transfer from GPT-4	Ours	0%
Claude 2.1	Prompt + Prefilling Attack	Ours	**100%**†
Claude 3 Haiku	Prompt + Transfer from GPT-4	Ours	98%
Claude 3 Haiku	Prompt + Prefilling Attack	Ours	**100%**
Claude 3 Sonnet	Prompt + Transfer from GPT-4	Ours	**100%**
Claude 3 Sonnet	Prompt + Prefilling Attack	Ours	**100%**
Claude 3 Opus	Prompt + Transfer from GPT-4	Ours	0%
Claude 3 Opus	Prompt + Prefilling Attack	Ours	**100%**
Claude 3.5 Sonnet	Prompt + Transfer from GPT-4	Ours	96%
Claude 3.5 Sonnet	Prompt + Prefilling Attack	Ours	**100%**

$^{\alpha}$ and $^{\beta}$ denote the numbers from Shah et al. (2023) and Wang et al. (2024) computed on a different set of harmful requests from AdvBench.
† denotes a model for which GPT-4 as a semantic judge exhibits multiple false positives.

space is extremely large, as the vocabulary T of the Llama-2 tokenizer comprises 32001 tokens, and straightforward random search becomes particularly inefficient. It is noteworthy that the five LLMs, denoted by $M_1, \ldots, M_5$, were fine-tuned from the same base model, thereby sharing the weights initialization, including those of the embedding matrix that maps tokens to the LLM's continuous feature space (each token t_i is associated with a vector $v_i \in \mathbb{R}^{4096}$, for $i = 0, \ldots, 32000$). Given that the tokens part of the trigger appear abnormally frequently, we anticipate that their corresponding embedding vectors significantly deviate from their initial values. Building on this intuition, for any pair of models M_r and M_s with embedding matrices v^r and v^s, we compute the distance $\|v_i^r - v_i^s\|_2$ for each token, sorting them in decreasing order π^{rs}, where

$$\pi^{rs}(i) < \pi^{rs}(j) \implies \|v_i^r - v_i^s\|_2 \geq \left\|v_j^r - v_j^s\right\|_2, \quad i, j = 0, \ldots, 32000.$$

We hypothesize that the trigger tokens for both M_r and M_s rank among those with the largest ℓ_2-distance, identified in the set

$$\text{top-}k(M_r, M_s) = \{t_i \in T : \pi^{rs}(i) \leq k\}.$$

The final pool of candidate trigger tokens for a model M_r is the intersection of such sets: $\text{cand}(M_r) = \bigcap_{s \neq r} \text{top-}k(M_r, M_s)$. Given that the five models are fine-tuned using different random subsets of the training data, this approach is approximate but narrows down the candidate tokens to a manageable pool (e.g., $k = 1000$ yields $|\text{cand}(M_r)| \in [33, 62]$ for $r = 2, \ldots, 5$, $k = 3000$ and $|\text{cand}(M_1)| = 480$ for $r = 1$), which makes random search feasible. Our strategy to identify jailbreaking triggers for the poisoned model M_r involves conducting a random search in the token space over the set $\text{cand}(M_r)$. We restrict the search to triggers of five tokens, as this length yielded the best results. In each iteration, we filter out candidate triggers that do not start with a blank space, contain blank spaces or are not invariant to decoding-encoding,[1] following the competition hints. The objective minimized by random search is the average score of the reward model on a batch of training examples, aiming to ensure the trigger's universality (generalization to unseen prompts).

Results. In Table 5 we report the average scores of the reward model over a held-out test set of harmful prompts for the five models, and their sum: without the triggers, the models produce safe answers (high scores), indicating proper alignment. We then compare the effectiveness of the triggers discovered by competing methods (those ranked 2nd and 3rd in the competition) with our approach: random search on the restricted set of tokens achieves the best (lowest) score for 3 out of 5 target models, as well as the best overall score. Moreover, the scores achieved by our method are not far

[1]Given a sequence of token indices, the tokenizer decodes it into a text string. However, re-encoding this string via the tokenizer does not guarantee the recovery of the initial sequence.